LIBSCHED : MODE D'EMPLOI

La bibliothèque d'ordonnancement `libsched` permet de tester des algorithmes d'ordonnancement de fonctions utilisateur. L'utilisateur a l'illusion que ses fonctions s'exécutent en parallèle.
Grâce à `libsched`, on peut définir et paramétrer de nouveaux algorithmes d'ordonnancement.
Deux fichiers sont fournis par la bibliothèque : `libsched.a` et le fichier d'inclusion `sched.h`

1. FONCTIONS DE LA LIBRAIRIE

```
#include "sched.h"

int CreateProc(function_t func, void *arg, int duration);
```

Cette fonction permet de créer une nouvelle fonction (que l'on appelle processus léger) qui pourra s'exécuter en parallèle avec d'autres. Le paramètre `func` est le nom de la fonction, `arg` est un pointeur vers les arguments de la fonction et `duration` est la durée estimée de la fonction. Par défaut le paramètre duration n'est pas utilisé mais il peut être utile pour des algorithmes d'ordonnnancement du type SJF (Shortest Job First).

`CreateProc` retourne l'identifiant du processus léger créé (`pid`).

```
void SchedParam(int type, double quantum, int (*felect)(void));
```

Cette fonction permet de régler les paramètres de l'ordonnanceur. `type` indique le type d'ordonnancement. 3 types sont possibles (définis dans `sched.h`) :

- BATCH indique un ordonnancement sans temps partagé de type FIFO. Dans ce cas, les paramètres `quantum` et `felect` sont ignorés.
- PREMPT indique un ordonnancement préemptif de type "tourniquet". C'est l'ordonnancement par défaut. Dans ce cas, le paramètre `quantum` est un décimal qui fixe la valeur du quantum de temps en secondes.
- NEW indique une nouvelle stratégie d'ordonnancement (définie par l'utilisateur). Dans ce cas, le paramètre `quantum` fixe la valeur du quantum de temps en secondes. Si `quantum` est égal à 0, l'ordonnancement devient non préemptif (sans temps partagé). Le paramètre `felect` est le nom de la fonction d'élection qui sera appelée automatiquement par la librairie avec une période de "`quantum` " secondes (si `quantum` est différent de 0).

La fonction d'élection `felect` doit avoir la forme suivante :

```
int Mon_election(void) {
/* Choix du nouveau processus élu */
    return elu;
}
```

La fonction d'élection choisit, parmi les processus à l'état `RUN`, le nouveau processus élu en fonction des informations regroupées dans la table `Tproc` définie dans `sched.h`:

```
struct proc {
    int flag;                       // Etat de la tâche : RUN, IDLE ou ZOMB
    int prio;                       // Priorité
    int pid;                        // Pid

...
    struct timeval end_time;        // date de fin
    struct timeval start_time;      // date de création
    struct timeval realstart_time;  // date de lancement
    double ncpu;                    // temps "cpu" consommé
    double duration;                // temps estimé de la tâche
} Tproc[MAXPROC];
```

La priorité d'un processus varie entre les deux constantes `MINPRIO` et `MAXPRIO`. Plus la valeur de la priorité est élevée, plus le processus est prioritaire.

Une fois le processus choisi, `felect` doit retourner l'indice dans `Tproc` du processus élu.

```
int GetElecProc(void);
```

Fonction qui retourne l'indice dans `Tproc` du processus élu (celui qu'on va décharger). Elle retourne –1 si aucun processus n'est élu.

```
void sched(int printmode);
```

Cette fonction lance l'ordonnanceur. L'ordonnancement effectif des processus ne commence qu'à partir de l'appel à cette fonction. Par défaut l'ordonnanceur exécute un algorithme similaire à Unix à base de priorité dynamique. Le paramètre `printmode` permet de lancer l'ordonnanceur en mode "verbeux ". Si `printmode` est différent de 0, l'ordonnanceur affichera à chaque commutation la liste des tâches prêtes. Cette fonction se termine lorsqu'il n'existe plus de tâche à l'état prêt (`RUN`).

```
void PrintStat(void);
```

Cette fonction affiche les statistiques sur les tâches exécutées (temps réel d'exécution, temps processeur consommé, temps d'attente).

2. EXEMPLE

L'exemple suivant illustre l'utilisation des primitives.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <malloc.h>
#include <sched.h>

// Fonction utilisateur

void MonProc(int *pid) {
    long i;

    for (i=0;i<8E7;i++)
        if (i%(long)4E6 == 0)
            printf("%d - %ld\n",*pid, i);

    printf("############ FIN PROC %d\n\n", *pid );
}
```

```
// Exemple de primitive d'election definie par l'utilisateur
// Remarques : les primitives d'election sont appelées directement
// depuis la librairie. Elles ne sont appelées que si au
// moins un processus est à l'etat pret (RUN)
// Ces primitives manipulent la table globale des processus
// définie dans sched.h

// Election aléatoire

int RandomElect(void) {
    int i;

    printf("RANDOM Election !\n");

    do {
        i = (int) ((float)MAXPROC*rand()/(RAND_MAX+1.0));
    } while (Tproc[i].flag != RUN);
    return i;
}


int main (int argc, char *argv[]) {
    int i;
    int *j;

    // Créer 3 processus

    for (i = 0; i < 3; i++) {
        j = (int *) malloc(sizeof(int));
        *j= i;
        CreateProc((function_t)MonProc,(void *)j, 0);
    }

    // Exemples de changement de paramètres

    // Définir une nouvelle primitive d'election avec un quantum de 1,5 secondes
    SchedParam(NEW, 1.5, RandomElect);

    // Redéfinir le quantum par defaut
    SchedParam(PREMPT, 2, NULL);

    // Passer en mode batch
    SchedParam(BATCH, 0, NULL);

    // Lancer l'ordonnanceur en mode non "verbeux"
    sched(0);

    // Imprimer les statistiques
    PrintStat();

    return EXIT_SUCCESS;
}
```

3. UTILISATION

La `libsched` est disponible sur le moodle de l'UE dans la Rubrique "Librairie TME".

Décompressez le fichier libsched.tgz

```
$ tar xvfpz libsched.tgz
```

Le sous-répertoire `demo` contient l'exemple précédent dans le fichier main.c avec un `Makefile` permettant de générer directement l'exécutable.
Le sous-répertoire `src` contient le source de `libsched`..

Il faut d'abord compiler la librairie :

```
$ cd libsched/src
$ make
```

Puis aller dans le répertoire demo, compiler et lancer l'exemple :

```
$ cd ../demo
$ make main
$ ./main
```